

Verifying Concurrent Programs by Timestamping Atomic Operations to Specify Fixed Linearization Points

Soo-Ho Park, Finn Hackett, Yuanhao Wei, & Ivan Beschastnikh
University of British Columbia

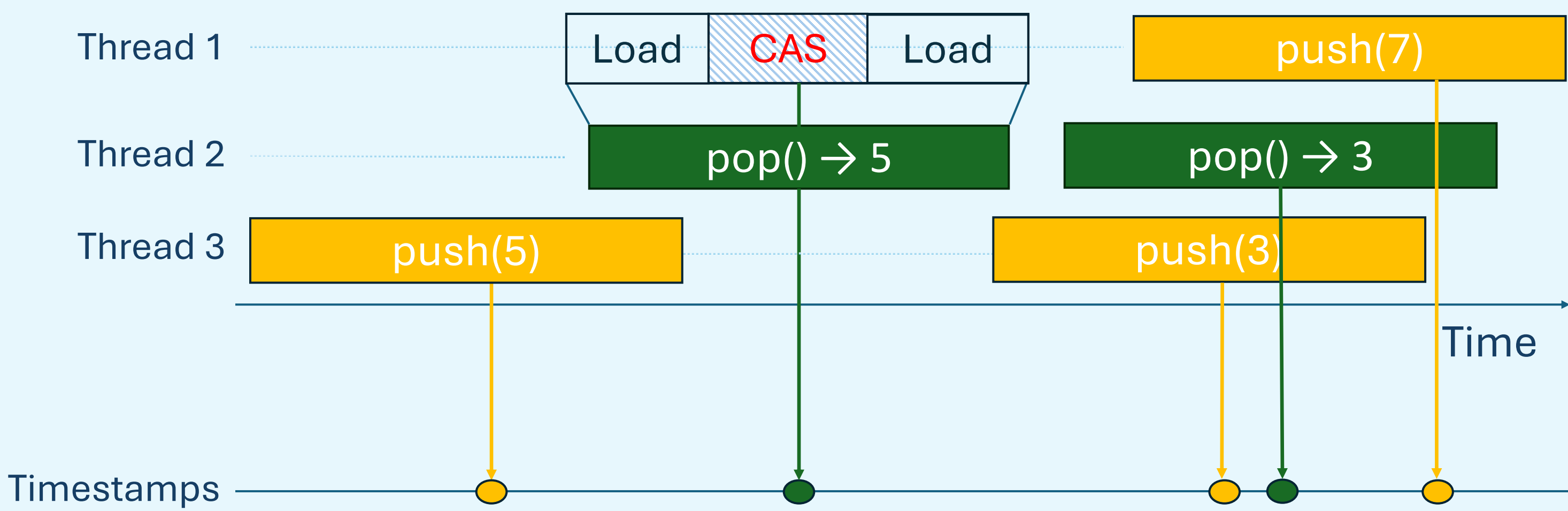
WHY ARE CONCURRENT SOFTWARE BUGS SO HARD TO FIND?

Software bugs are costly and difficult to diagnose. Ensuring correctness in concurrent software is particularly challenging because program behavior must remain correct across all possible nondeterministic interleavings of operations.

Atomic operations ensure that their effects are indivisible, but how can we ensure that high-level operations built from these atomic ops are functionally correct? Their observed results are consistent with some valid sequential ordering of the operations?

OUR APPROACH

Assign timestamps to atomic **Load**, **Store**, and **CAS** operations. These timestamps can be used to specify linearization points within higher-level operations and establish a total order among them.

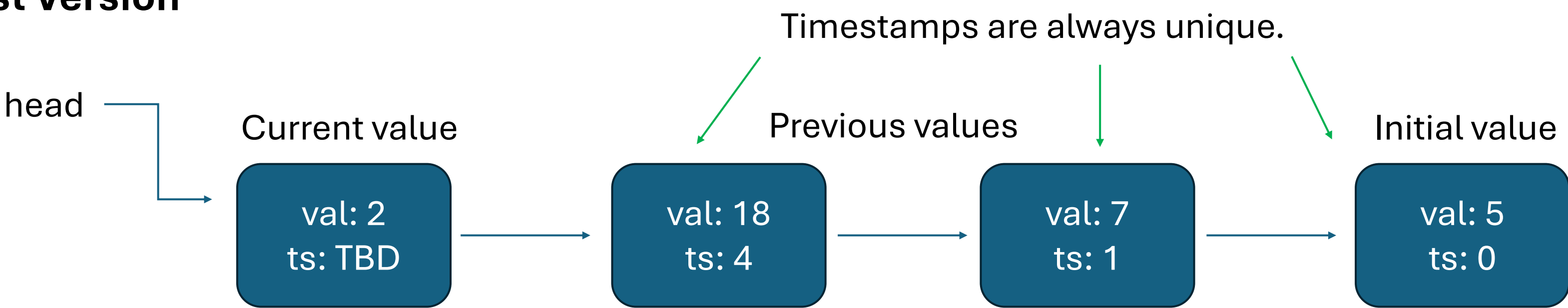


Timestamping Atomic Load, Store, and CAS operations

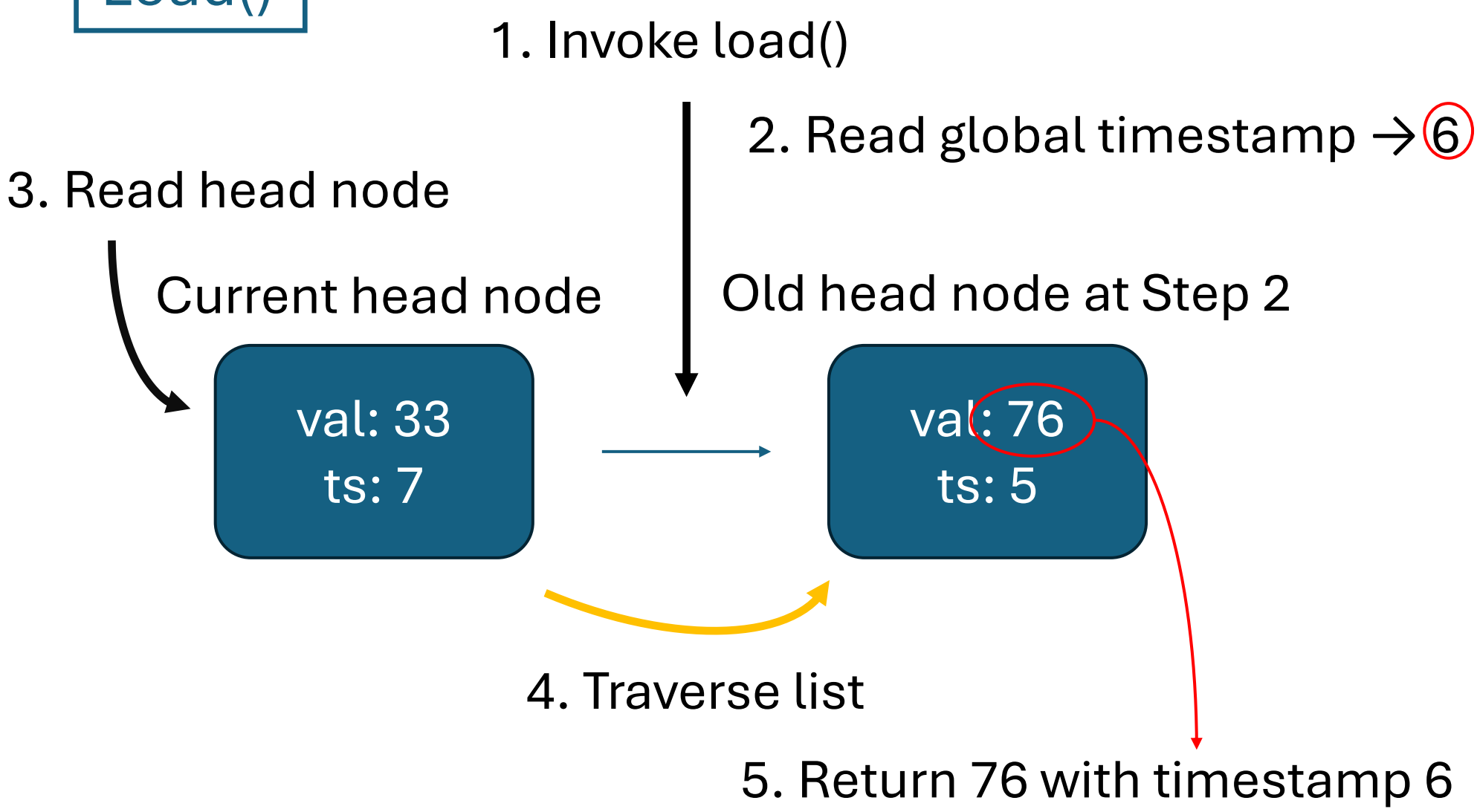
GitHub repository for codebase and proof of correctness



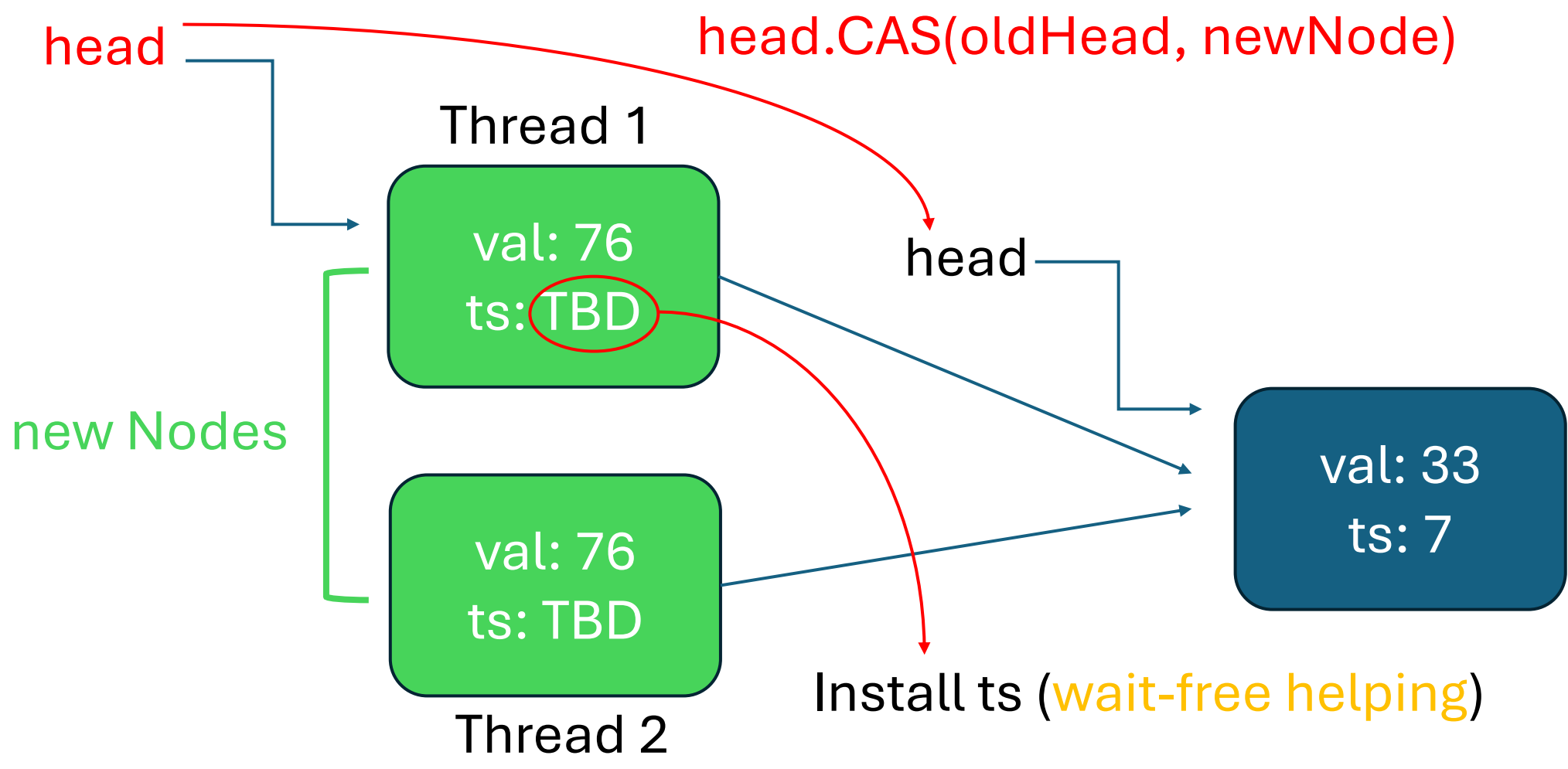
V0: Linked-List Version



Load()



Store()



V1: Single Node Version

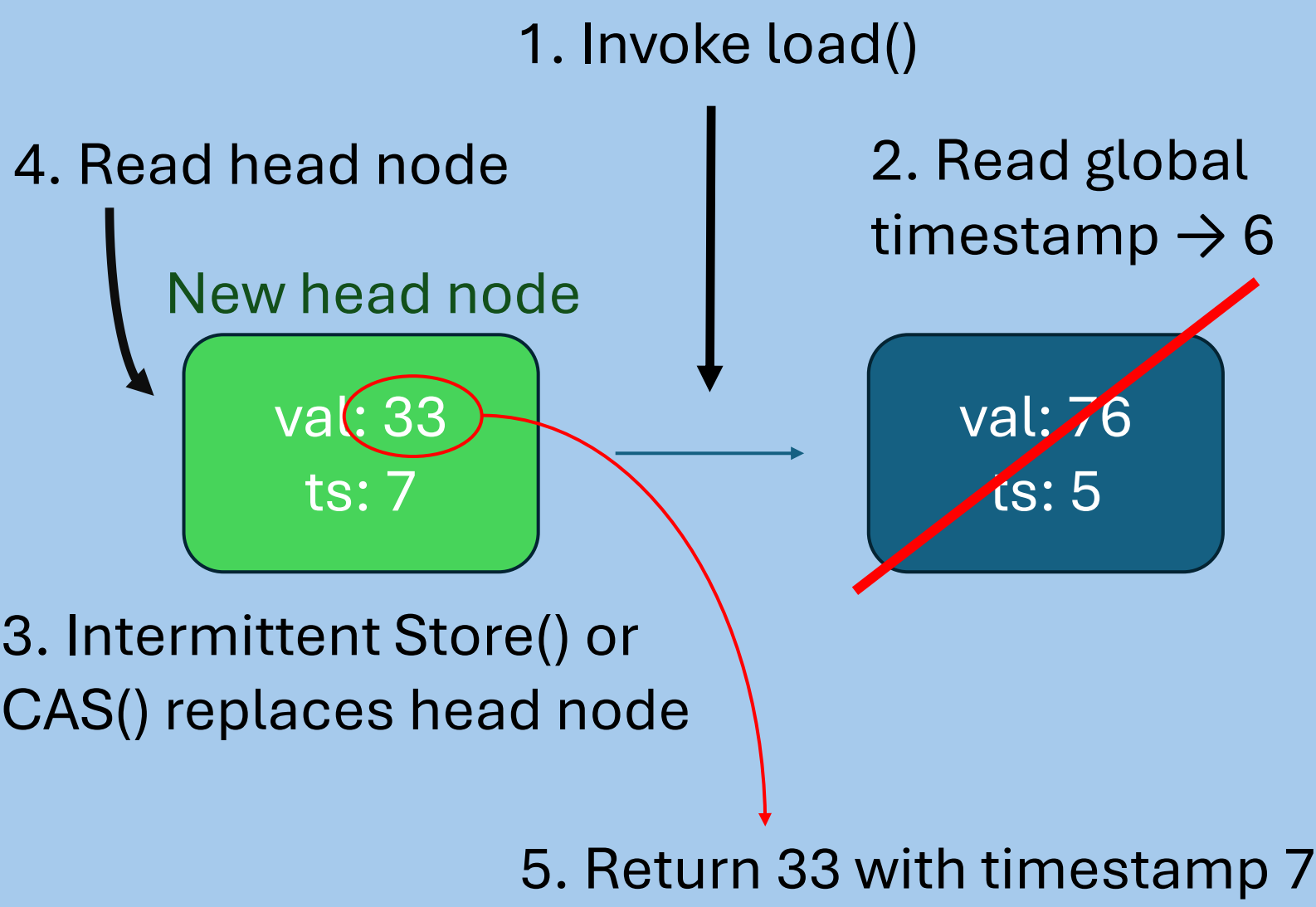
Maintains the **most recent** node with the timestamp of its installation

Only one thread can modify a node through the **head pointer** at a time.

EBR reclamation strategy for safe node deallocation

Load()

Load() returns the **maximum** of its global ts read and the ts of the head node



Future Work:

- PERFORMANCE STATISTICS
- Implement V2 where there is no indirection to reach the node
- Prove correctness for V1 (V0's proof has been done)
- Add timestamping for other atomic instructions such as fetch_add and fetch_sub
- Explore methods for specifying non-fixed linearization points

